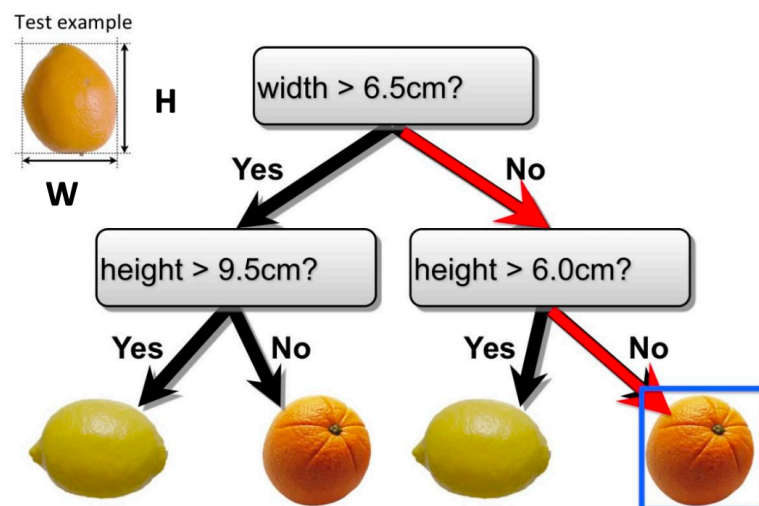


Decision Trees

Idea

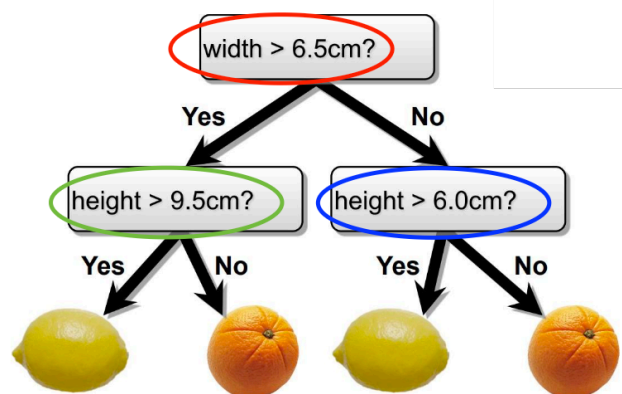
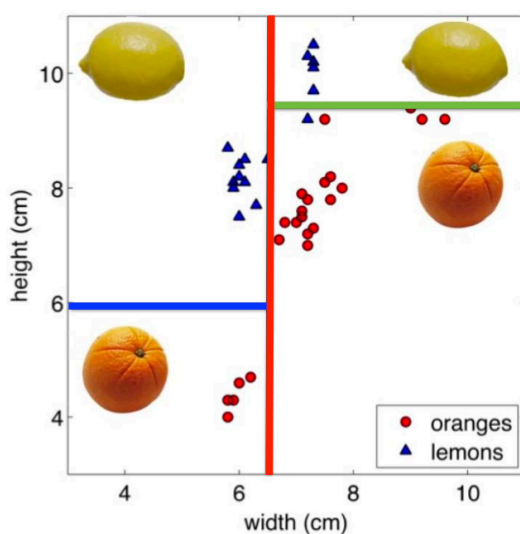
Make **predictions by recursively splitting on different attributes** → tree structure

- **Internal nodes:** test attributes
- **Branching**
- **Leaf nodes:** resulting predictions



Trees **can be represented in an orthogonal space:**

- **Thresholds** are the **boundaries parallel to the axes**



Continuous and discrete attributes

They can handle both continuous and discrete attributes

- Heterogeneous trees can be built

Continuous attributes

The **splitting** is based on a threshold

- Less than or greater than the threshold branching

Discrete attributes

The **splitting** is based on the attribute value itself

Defining the split attributes

Given a training set, we need to learn:

- Which attribute to choose for splitting
- What threshold to use for splitting

Each path from root to leaf defines a region R_m of input space

- Training examples that fall into the region R_m have the form

$$\{(x^{(m_1)}, t^{(m_1)}, \dots, x^{(m_k)}, t^{(m_k)})\}$$

(Where x is a feature, t is the associated label)

Classification tree

- Discrete output
- Leaf value y^m : most common value in $\{t^{(m_1)}, \dots, t^{(m_k)}\}$

$$y^m = \arg \max_{t \in \{1, \dots, C\}} \sum_{i=1}^k \mathbf{1}(t = t^{(m_i)})$$

(the function $\mathbf{1}$ equals to 1 if $t = t^{(m_i)}$)

Regression tree

- Continuous output
- Leaf value y^m : mean value in $\{t^{(m_1)}, \dots, t^{(m_k)}\}$

Learning a decision tree

Learning the simplest (smallest) decision tree that correctly **classifies a training set** is a **NP-complete problem**

→ We **must rely on a greedy heuristic**

- **Start** with: **empty decision tree**, **complete training set**
 - **Split** on the **best attribute**
 - **Apply recursively** on the **subpartitions**

In the following measurements:

- A region R is given and is **split into two regions** R_1 and R_2
- **We want to measure the level of change before and after the split**
 - → **Different splits** lead to **different findings**

Accuracy gain

Measures the **accuracy of a prevision**

The misclassification error of R is:

- Before the split: $L(R)$
- After the split: $L(R_1) + L(R_2) = \frac{|R_1|}{|R|}L(R_1) + \frac{|R_2|}{|R|}L(R_2)$

$\frac{|R_i|}{|R|}$ is the ratio of samples on the i -th split

The **accuracy gain** is:

$$AG = L(R) - \frac{|R_1|L(R_1) + |R_2|L(R_2)}{|R|}$$

Information gain

Measures the **expected reduction in entropy**

Entropy is calculated as

$$H(X) = - \mathbb{E}_{X \sim p} [\log_2 p(X)] = - \sum_{x \in X} p(x) \log_2 p(x)$$

It **measures** the **expected “surprise”**:

- **High** entropy → **values** are **less predictable**
- **Low** entropy → **values** are **more predictable**

The entropy of R is:

- Before the split: $H(R)$
- After the split: $H(R_1) + H(R_2) = \frac{|R_1|}{|R|} H(R_1) + \frac{|R_2|}{|R|} H(R_2)$

$\frac{|R_i|}{|R|}$ is the ratio of samples on the i -th split

The **information gain** is:

$$IG = H(R) - \frac{|R_1| H(R_1) + |R_2| H(R_2)}{|R|}$$

Gini Index

Measures the **“impurity” at a given node**

- **Pure node**: **all** the **samples** in the node **belong to the same region** R → it's a **leaf**

$$\text{GINI}(R) = 1 - \sum_j [p(j|R)]^2$$

Where $p(j|R)$ is the **class frequency of class j in region-node R**

- Aims to measure the **probability of misclassifying a randomly chosen element**
 - **Greater** value → **greater chance of misclassification**

We choose attributes and values that:

- Highest accuracy gain
- Highest information gain
- Lower GINI index

Pruning

When building a decision tree **we should respect** the principle of the **Occam's razor**

- The simplest explanation is usually the right one
- We should prefer simplest trees compared to more complex ones

Pruning allows to reduce complexity

- Replaces a subtree by a leaf node

Problems

Decision trees have the following **disadvantages**:

- Exponentially **less data at lower levels**
- **Mistakes** are **propagated** down the tree

They have a **high risk of overfitting**:

- Their structure is determined by the data → easily fit the training set

Random Forest

The **main problem** of decision trees is the **high variance** (= overfitting)

We can **apply an ensemble model**:

- The **predictions of multiple simple models** are **converged** in **one single prediction**
- **Random Forest** is an ensemble model, as it **trains multiple decision trees**

It applies two main techniques: bagging with bootstrapping and feature randomness

Bagging and Bootstrapping

A set number n of **decision trees are trained**:

- **Each** on a **random subset of the original training set** → created **via bootstrapping**
 - An example is sorted from the dataset
 - It is added to the new subset
 - The example is reinserted in the dataset, for future sorting
- Allows **to lower the variance**
- → The **result** is the **mean value** (regression) or the **majority vote** (classification)

Feature Randomness

Each split node can only **apply** a **random subset of features**

- To obtain **decorrelated decision trees** in bagging
- **Usually** $m = \sqrt{p}$ features are chosen